FlowMES — Complete Project Documentation

The whole story, end to end: why it was built, what it does, how it works technically, and what every single code file is for — written so a non■programmer can follow along.

> **One■line summary:** FlowMES is a web■based "control tower" for a factory. It watches every machine in real time, calculates how efficiently the factory is running (a score called **OEE**), tracks work orders, inventory, quality and maintenance, and warns managers before things go wrong — all through a browser, with no expensive hardware required.

Table of contents

Part 1 — The Story: why FlowMES was built

1.1 The problem

Walk into a **small or mid■size factory** (an "SME manufacturer") almost anywhere in the world and you'll find the same thing: the machines are modern, but the way the factory is *managed* is not. Production is tracked on:

- **Whiteboards** wiped clean at the end of every shift,
- **Excel spreadsheets** that only one person understands,
- **Paper job cards** that get lost, and
- **WhatsApp groups** where the supervisor asks "is CNC■2 running?"

Because of this, the factory owner cannot answer basic questions:

- *How much of my machines' time is actually productive today?*
- *Which machine breaks down most, and what is it costing me?*
- *Do I have enough steel to finish this week's orders?*
- *Why did we miss the delivery to our biggest customer?*

Large corporations solve this with software called an **MES — a Manufacturing Execution System**. But traditional MES software (from vendors like SAP or Siemens) costs **hundreds of thousands of dollars**, takes a year to install, and needs a dedicated IT team. That puts it completely out of reach for the SME factories that make up the backbone of manufacturing.

1.2 The idea

FlowMES exists to close that gap: an *enterprise■grade* MES that an SME can actually afford and switch on in days, delivered as a website (Software■as■a■Service) instead of a giant on■premise installation.

The wedge — the specific first customer the product was shaped around — is **GMATS Machineries**, a compressor manufacturer in Bengaluru, India, who provided a detailed real■world spec for how their inventory and sales process works. That spec became the **GMATS inventory module** (see Part 3.9), which proves FlowMES can model a real client's exact workflow rather than a generic textbook one.

FlowMES is the flagship product of **MARX8** (marx8.com), an industrial■technology studio. FlowMES is the "Manufacturing Execution" pillar of that brand.

1.3 What makes it different

Traditional MES	FlowMES
Costs $100k+	SaaS subscription an SME can afford
6–12 month installation	Live in days
Needs an IT team	Runs in a browser
Needs expensive PLC/hardware wiring	Works with simulated data now; connects to real PLCs via a lightweight edge agent later
Generic	Modelled around a real client's actual process (GMATS)

1.4 How it was built

FlowMES was developed **solo, in ~30 incremental "phases"** — you'll still see that history in the code (files and TypeScript types named `phase8`, `phase27`, `mega-pack1`, etc.). Each phase added one slice of functionality (machines -> OEE -> work orders -> inventory -> quality -> predictive AI -> IoT -> multi■tenancy). This is why the frontend has ~50 self■contained "section" components: each is one module bolted on in one phase.

Part 2 — What FlowMES actually does (in plain English)

Think of FlowMES as **five layers of usefulness stacked on top of each other.**

2.1 Layer 1 — See the factory (Core MES)

The foundation is **real■time visibility**. FlowMES knows, right now:

- Which machines are **Running / Idle / in Breakdown / under Maintenance**
- How **busy** each machine is (utilization %)
- How much **downtime** has piled up and *why* (broken belt? tool change? power cut?)
- What each **shift** produced versus its target

The headline number it calculates is **OEE — Overall Equipment Effectiveness**. This is the single most important score in manufacturing. It answers: *"Of the time this machine could have been making good parts, what fraction did it actually spend making good parts?"* It's the product of three simpler scores:

```
OEE  =  Availability   ×   Performance      ×   Quality
       (was it on?)       (did it run fast    (were the parts
                           enough?)            good?)
```

A machine that's on 90% of the time, running at 91% of its ideal speed, making 97% good parts has an OEE of `0.90 × 0.91 × 0.97 ≈ 79%`. World■class is ~85%. FlowMES computes this continuously for every machine and for the whole factory.

2.2 Layer 2 — Run the factory (Operations)

- **Work Orders** — "make 500 of part SHAFT■001 on CNC■01." FlowMES tracks each order from *Planned -> In Progress -> Completed*, and — cleverly — when an order finishes it **automatically deducts the raw material used and adds the finished goods to inventory** (this is the "BOM movement", explained in 3.10).
- **Production Planning & Scheduling** — what runs on which machine, on which shift, on which day.
- **Operator Terminal** — the screen a machine operator uses to log the job they're running and how many good/rejected parts they made.
- **Orders & Dispatch** — customer orders and how much has shipped.

2.3 Layer 3 — Improve the factory (Manufacturing Intelligence)

- **Executive OEE dashboards** — the boss's view.
- **Predictive Maintenance** — FlowMES scores each machine's *risk of failing soon* based on its downtime history, breakdown frequency, reject rate and workload, and recommends action *before* it breaks.
- **Smart Alerts & Escalations** — automatically raises a flag when OEE drops, a machine breaks down, quality slips, or stock runs low, and turns those flags into trackable "escalations" assigned to a person.
- **AI Factory Copilot** *(optional)* — a chat assistant you can ask *"why is my OEE low today?"* or *"what should I reorder?"* and it answers using your real factory data. It also writes a daily management report.

2.4 Layer 4 — Support the factory (Factory Operations)

- **Inventory** (raw materials, tooling, consumables, finished goods, spares) with automatic low■stock alerts.
- **Purchasing** (suppliers + purchase orders).
- **Quality** (inspections, pass/fail, defects, rework, scrap).
- **Maintenance (CMMS)** (preventive/corrective tasks).
- **Documents** (SOPs, quality plans, compliance records).
- A full **GMATS enterprise inventory** module modelling a real compressor company's quote■to■invoice process.

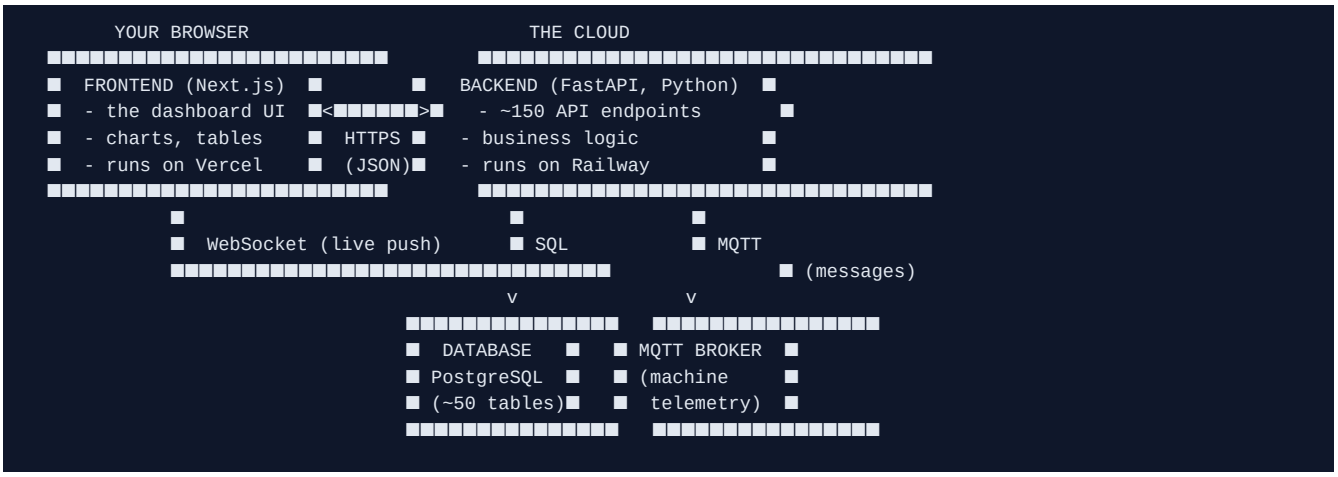
2.5 Layer 5 — Connect & sell the factory (IoT + Platform)

- **Industrial IoT / Connectivity** — the layer that talks to the physical machines. Today it *simulates* live PLC (machine■controller) signals; the architecture is ready to plug into real machines over industrial protocols (OPC UA, Modbus, Siemens S7, Allen■Bradley, Beckhoff, Omron).
- **Multi■tenancy & white■label** — one FlowMES installation can serve many companies at once, each seeing only its own data, its own logo and its own colour scheme, on its own subscription plan. This is what makes it a *SaaS product*, not just one factory's tool.
- **Roles** — every user is an **Admin**, **Supervisor** or **Operator**, and only sees/does what their role allows.

Part 3 — How it works technically

3.1 The big picture

FlowMES is split into two programs that talk over the internet, plus a database and a real■time messaging layer:



- The **Frontend** is what you see — the dashboard. It holds no business logic; it just asks the backend for data and draws it.
- The **Backend** is the brain — it holds all the rules, does the OEE math, enforces permissions, and reads/writes the database.
- The **Database** is the memory — every machine, order, part and inspection is a row in a table.
- The **MQTT broker** is a real■time "post office" that machines use to shout their status; the backend listens and reacts instantly.

3.2 The technology stack (and why)

Layer	Technology	What it is / why
Frontend framework	**Next.js 16 + React 19**	Builds fast, modern web pages.
Frontend styling	**Tailwind CSS 4**	A shorthand for styling without writing separate CSS files.
Frontend language	**TypeScript**	JavaScript with type■checking, so bugs are caught before running.
Backend framework	**FastAPI (Python)**	A high■speed web framework that auto■generates API docs.
Database toolkit	**SQLAlchemy**	An "ORM" — lets Python talk to the database using objects instead of raw SQL.
Database	**PostgreSQL** (cloud) / **SQLite** (local)	Where all data lives. Postgres in production, a simple file locally.
Real■time (machines)	**MQTT (paho■mqtt)**	Lightweight messaging protocol used across all of industrial IoT.
Real■time (browser)	**WebSockets**	A permanent open pipe so the server can *push* updates to the dashboard instantly.
Login security	**JWT (python■jose) + bcrypt**	Tokens to prove who you are; bcrypt to store passwords safely.
AI (optional)	**Anthropic Claude API**	Powers the natural■language copilot.
Hosting	**Vercel** (frontend) + **Railway** (backend)	Push code to GitHub -> it deploys itself.

3.3 What happens when you log in (the request lifecycle)

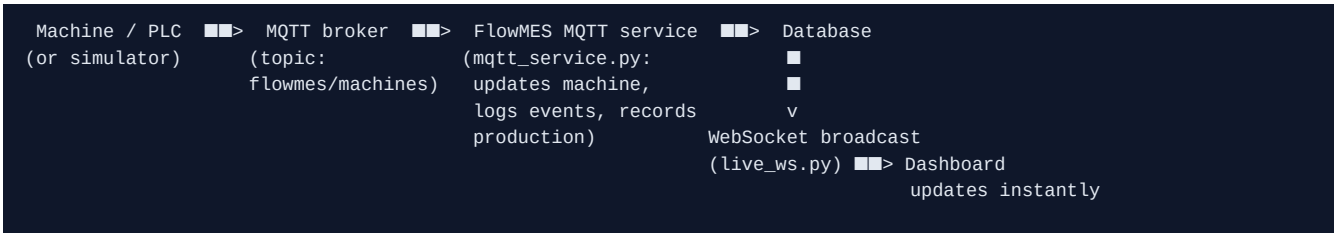
1. You type a username + password on the **login page** and hit *Login*.
2. The frontend sends them to the backend endpoint `POST /login`.
3. The backend looks up the user, checks the password with **bcrypt**, and if correct, mints a **JWT** — a digitally■signed token that encodes *who you are* (`sub`), *your role* (`Admin`/`Supervisor`/`Operator`) and *your company* (`tenant`). The token expires in 4 hours.
4. The frontend stores that token in the browser and attaches it to **every** future request (`Authorization: Bearer <token>`).
5. Every protected endpoint checks the token before doing anything. Some endpoints additionally require a specific role (e.g. deleting things is Admin■only).

Because the token carries your *company* and *role*, the backend can automatically show you only your company's data and hide buttons you're not allowed to use — no extra lookups needed.

3.4 How "real■time" actually works (two engines)

FlowMES stays live through **two independent mechanisms**:

Engine 1 — the real telemetry pipeline (MQTT -> WebSocket). This is the "production" path meant for real machines:



A machine publishes a small JSON message like `{"machine":"CNC-01","status":"Running","utilization":87,...}`. The backend's embedded MQTT service catches it, updates that machine's row, logs a status■change event if the status changed, saves a production record, and then **pushes** the update to every open dashboard through a WebSocket — so the screen changes without the user refreshing.

Engine 2 — the simulation loop (keeps the demo alive). Because there are no physical machines connected in the demo, a background task inside the backend (`_simulation_loop` in `main.py`) wakes up **every 45 seconds** and gently nudges the factory: it advances a work order, pushes new IoT signals, adds a production record (so OEE trends keep moving), occasionally flips a machine's status, consumes some inventory, logs quality checks, etc. This is what makes the live demo look like a breathing factory even with nobody touching it. On top of that, a separate PLC simulator (`phase30_plc_simulator.py`) can publish real MQTT telemetry through Engine 1.

3.5 The database (the factory's memory)

Everything is stored in **~50 tables** defined in `models.py`. Grouped:

- **Shop floor:** `machines`, `downtime_logs`, `machine_events`, `production_records`, `shift_data`
- **Operations:** `work_orders`, `production_plans`, `production_schedules`, `customer_orders`, `operator_job_executions`
- **Materials:** `inventory_items`, `inventory_transactions`, `suppliers`, `purchase_orders`, plus advanced stores logistics (`remnants`, `material_issue_slips`, `goods_receipt_notes` + `grn_items`, `cycle_counts` + `cycle_count_items`)
- **Quality & maintenance:** `quality_inspections`, `maintenance_tasks`

- **Intelligence:** `alerts`, `escalations`, `ai_recommendations`, `notifications`, `iot_telemetry`, `cost_records`, `report_requests`
- **Layout & docs:** `factory_layout_nodes`, `compliance_documents`
- **Industrial IoT:** `industrial_devices`, `industrial_signals`, `plc_signal_mappings`
- **The SaaS platform:** `users`, `tenant_configs` (licence + branding per company), `company_tenants` (billing), `audit_logs`
- **GMATS client module:** `gmats_items`, `gmats_aliases`, `gmats_proformas` + `_lines`, `gmats_invoices`, `gmats_min` + `_lines`

SQLAlchemy turns each table into a Python class, so the code says `db.query(models.Machine)` instead of writing raw SQL. `Base.metadata.create_all()` at startup creates any missing tables automatically.

3.6 Multi■tenancy — one app, many companies

This is the feature that makes FlowMES a *product* rather than one factory's tool. The key is a single text field: **`tenant_code`**.

- **`DEFAULT`** is the founder/internal workspace — a super■admin who can switch between companies and see everything.
- A **named tenant** like **`GMATS`** is a real client — locked to its own data, its own branding, its own subscription.

When a client logs in, their `tenant_code` is baked into their JWT. The backend derives the tenant *from the token*, never from what the browser claims, so one client can never peek at another's data. The `TenantConfig` table stores, per tenant: which **module packs** are unlocked (licensing), the **brand name/colour/logo** (white■label), and the **subscription/trial** state (billing). `platform_routes.py` serves all of this.

3.7 Roles & permissions (RBAC)

Every user is one of three roles, enforced on **both** sides:

Role	Can do
Operator	Update/log work on the shop floor (their terminal, machine status, quality entry).
Supervisor	Everything an operator can, plus create/approve records across most modules.
Admin	Everything, including deleting records and managing users.

- **Backend enforcement** (`auth.require_roles([...])`) is the real security — it rejects unauthorised API calls no matter what.
- **Frontend enforcement** (`modules.ts` -> `canRoleSeeView`) is the UX — it hides menu sections and buttons a role shouldn't see, so an Operator's screen looks appropriately simple.

3.8 How OEE and predictive risk are calculated

OEE (`analytics_engine.calculate_oee_from_record`): for each production record, `Availability = runtime ÷ planned time`, `Performance = (ideal cycle time × parts made) ÷ runtime`, `Quality = good parts ÷ total parts`, and `OEE = A × P × Q`. Simple arithmetic, but it's the heartbeat of the whole system.

Predictive risk (`predictive_engine.calculate_predictive_risk`): each machine starts at 0 and *accumulates points* for warning signs — currently broken (+35), low utilization (+20), lots of accumulated downtime (+25), frequent

breakdowns (+20), high reject rate (+20), heavy work■order backlog (+10), etc. The total (capped at 100) becomes a **risk level** (Low/Medium/High/Critical) with a plain■English recommendation. It's a transparent rules■based model — no black box — which is exactly right for a factory manager who needs to trust and act on it.

3.9 The GMATS inventory lifecycle (a real client's workflow)

GMATS sells compressors *and* spare parts, and their stock has to be tracked as **four buckets**:

```
 Physical  =  what's actually on the rack
Reserved  =  blocked by open customer quotations (proformas)
Available =  Physical − Reserved   (what you can still promise)
Reorder   =  minimum before you must re-purchase
```

The real sales flow, modelled exactly, is:

```
 Stock■In (+physical)
     ■
     v
Proforma quote  ■■>  reserves stock (Reserved ↑, Available ↓)
     ■
     ■■> Tax Invoice   ■■>  deducts Physical, clears the Reserve (a real sale)
     ■
     ■■> Material Issue Note (MIN) ■■> free spare parts shipped with a machine
                                       (deducts Physical, not billed)
     ■
     v
Reorder alert when Available ≤ Reorder level
```

It also supports **item aliases** (one part known by many names) and **admin "undo"** actions to correct an operator's mistake (void an invoice to restore stock, void a MIN, or directly correct the counts). All of it is tenant■scoped so it only ever touches GMATS's own data. This lives in `gmats_inventory_routes.py`.

3.10 Work■order BOM automatic material movement

A **BOM (Bill of Materials)** is the recipe for a product — e.g. one SHAFT■001 needs 2 kg of steel. In `main.py` there's a `PART_BOM` recipe book. When a work order is marked **Completed**, FlowMES automatically:

1. **Deducts** the raw material used (`quantity × consume_per_unit`) from inventory, and
2. **Adds** the finished goods produced,

each recorded as an inventory transaction. So finishing a job of 5 shafts turns "10 kg steel" into "5 finished shafts" in the stock ledger — no manual data entry. This is exactly the kind of automation that makes an MES worth having.

3.11 The AI Factory Copilot

An **optional** assistant (`ai_copilot.py`). It's **off** unless an `ANTHROPIC_API_KEY` is set in the server's environment — so it costs nothing until a client wants it. When on, it:

1. Builds a compact text snapshot of the factory (machines, average OEE, recent downtime, shift outputs, low stock),
2. Sends that plus the user's question to **Claude** (Anthropic's AI) over a plain HTTPS call (no heavy SDK, so it can never break the deployment),
3. Returns a concise, practical answer grounded *only* in the real data — plus a one■click daily management report.

3.12 Industrial connectivity (the bridge to real machines)

`industrial_adapters.py` defines a small **adapter framework**: a registry of supported protocols (OPC UA, Modbus TCP, Siemens S7, Allen■Bradley, Beckhoff ADS, Omron FINS) and a `SimulatorAdapter` that produces realistic values for each *without any hardware*. Every 45 seconds the simulation loop "polls" one device through its adapter and stores the readings as live `industrial_signals`. To go live on a real floor, you swap the simulator for a real driver on a small on■site "edge agent" — everything downstream (signals, dashboards) stays identical.

3.13 How it's deployed

- **Backend -> Railway.** Push to the `master` branch on GitHub and Railway rebuilds it using **NIXPACKS** (config in `railway.toml`), starting the FastAPI app with `uvicorn`. Its health is checked at `/docs`.
- **Frontend -> Vercel.** Push to `master` and Vercel rebuilds the Next.js site. Production lives at `flow-mes.vercel.app`.
- **Config via environment variables** (never hard■coded): database URL, secret key, MQTT broker, allowed origins (CORS), the optional AI key, the optional error■monitoring key, and the GMATS admin password.

Part 4 — Every code file explained

> Reading guide: each file gets a **plain■English** line (what it's *for*) and a **technical** line (how it does it). Files are grouped by area.

4.1 Backend — the core application

`backend/main.py` — the heart of the backend (~3,600 lines)

- **Plain English:** The main switchboard. Almost every button the dashboard presses lands here. It defines the ~150 web addresses (endpoints) for machines, work orders, inventory, quality, analytics, reports, and everything else, and it starts the real■time and simulation engines when the server boots.
- **Technical:** Creates the `FastAPI` app; creates DB tables; runs a tiny self■migration to add `users.tenant_code`; optionally enables Sentry; **registers** the add■on modules (`enterprise_inventory_routes`, `gmats_inventory_routes`, `platform_routes`, `ai_copilot`, `industrial_adapters`); on startup launches the **MQTT service** and the async `_simulation_loop` (45 s ticks) and seeds demo data + the `gmats` login; locks down **CORS** to known origins; defines `VALID_ROLES`, the `CLIENT_TENANTS` map, the `get_db` dependency, and the OEE/alert helper functions; then defines the huge REST surface (each resource has list/create/update/delete + an `/analytics/...` summary). Also holds `PART_BOM` and the work■order completion logic that auto■moves inventory.

`backend/database.py` — the database connection

- **Plain English:** Opens the connection to the database and hands out a fresh "session" for each request.
- **Technical:** Reads `DATABASE_URL` from the environment, builds the SQLAlchemy `engine` (with `pool_pre_ping` to survive dropped connections), the `SessionLocal` factory, and the `Base` class every model inherits from. `get_db()` yields a session and always closes it.

`backend/models.py` — the shape of all data (~50 tables)

- **Plain English:** The blueprint of the factory's memory. Every kind of thing FlowMES remembers — a machine, an order, a part, an inspection, a user, a company — is described here as a table.
- **Technical:** ~50 SQLAlchemy ORM classes (each `= one table`), with columns, defaults, foreign■key links and a few relationships. Includes the core MES tables, the SaaS/platform tables (`User`, `TenantConfig`, `CompanyTenant`, `AuditLog`), the industrial■IoT tables, and the tenant■scoped GMATS inventory tables.

`backend/schemas.py` — the API's data contracts

- **Plain English:** Defines exactly what data each endpoint expects to receive and what it will send back, so bad data is rejected politely.
- **Technical:** Pydantic models (e.g. `WorkOrderCreate`, `WorkOrderResponse`, `QualityInspectionCreate`). FastAPI uses these to validate request bodies and to shape/validate responses (`response_model=...`).

`backend/auth.py` — who are you? (identity)

- **Plain English:** Issues the "ID badge" (a token) when you log in and checks it on every request, and enforces role permissions.
- **Technical:** `create_access_token` signs a JWT (HS256, 4■hour expiry) with `SECRET_KEY`; `verify_token`/`get_current_user` decode it; `require_roles([...])` is a dependency that returns 403 unless the caller's role is allowed.

`backend/security.py` — password safety

- **Plain English:** Scrambles passwords so even the people running the database can't read them, and quietly upgrades old accounts to the stronger method.
- **Technical:** `hash_password`/`verify_password` use **bcrypt** (truncating to bcrypt's 72■byte limit); detects **legacy SHA■256** hashes and verifies them too; `needs_rehash` flags those for transparent upgrade on next login.

4.2 Backend — real■time & simulation

`backend/mqtt_service.py` — the machine listener

- **Plain English:** Sits with an ear to the "machine radio" (MQTT). When a machine reports its status, this saves it to the database and instantly pushes the change to every open dashboard.
- **Technical:** An embedded paho■mqtt client running on a background thread subscribes to `flowmes/machines`; `on_message` parses the JSON payload, upserts the `Machine`, logs a `MachineEvent` on status change, writes a `ProductionRecord` when a batch completes, logs downtime on breakdown, and calls `safe_broadcast` -> the WebSocket manager.

`backend/live_ws.py` — the live push pipe

- **Plain English:** Keeps a permanent open line to every dashboard so the server can push updates the instant they happen.
- **Technical:** A `ConnectionManager` tracking active `WebSocket` connections with `connect`/`disconnect`/`broadcast` (JSON), plus `broadcast_live_event`. `main.py` exposes the socket at `GET /ws/live`.

`backend/factory_simulator.py` — the demo factory generator

- **Plain English:** Two jobs: (1) on first run, fill an empty database with a believable Indian SME factory (5 machines, suppliers, 24 inventory items, work orders, shifts, quality, maintenance, costs, IoT, etc.); (2) provide the "tick" functions that keep that factory moving.
- **Technical:** `seed_all()` calls idempotent `_seed_*` helpers (each skips if data exists). `tick_*` functions (`tick_work_order_progress`, `tick_quality`, `tick_iot`, `tick_inventory`, `tick_production`, `tick_machine_status`, …) each perform one small realistic mutation; `run_simulation` picks a weighted random handful per tick. Can also be run standalone (`python factory_simulator.py`).

`backend/analytics_engine.py` — the maths brain

- **Plain English:** Turns raw numbers into the meaningful scores managers care about — OEE, shift efficiency, the executive summary, and smart alerts.
- **Technical:** `calculate_oee_from_record` (A×P×Q), `build_shift_kpis`, `build_oee_trends`, `build_management_summary` (top loss reason, worst machine, downtime cost, target achievement), and `build_smart_alerts` (breakdown / low■utilisation / low■OEE / high■reject / downtime■escalation rules).

`backend/predictive_engine.py` — the "will it break?" scorer

- **Plain English:** Gives each machine a health■risk score and a recommendation, so you fix things before they fail.
- **Technical:** `calculate_predictive_risk` aggregates downtime, breakdown transitions, reject rate and work■order pressure into an additive score (capped 100), classified into Low/Medium/High/Critical with a text recommendation, sorted worst■first.

`backend/report_generator.py` — the report writer

- **Plain English:** Formats the daily factory summary into clean readable text.
- **Technical:** `build_daily_summary_text(summary, shift_kpis, alerts)` returns a formatted plain■text report (used by `GET /reports/daily-summary.txt`).

4.3 Backend — add■on route modules (each `register(app)`ed by main.py)

`backend/platform_routes.py` — the SaaS plumbing

- **Plain English:** The behind■the■scenes business layer: which company gets which features, their branding, their subscription, the audit trail of who did what, and a health check for uptime monitoring.
- **Technical:** `TenantConfig` CRUD (`/tenant-config`, `/tenant-configs`), `log_audit()` + `/audit-logs`, `get_or_create_config` (30■day trial defaults), `seed_tenant_configs`, and a public `GET /health`. DEFAULT■tenant admins can license/brand any client; client admins can only re■brand themselves.

`backend/gmats_inventory_routes.py` — the GMATS client module

- **Plain English:** The exact quote■to■invoice inventory workflow of the first real client (GMATS compressors): four stock buckets, quotations that reserve stock, invoices that sell it, free■spare issue notes, item aliases, and admin undo.
- **Technical:** Tenant■scoped endpoints under `/gmats/*` using `_effective_tenant`/`_guard_record` to prevent cross■tenant access. Models the Physical/Reserved/Available/Reorder buckets, proforma reservation, tax■invoice deduction, Material Issue Notes, alias lookup, CSV import, and void/correct operations (with careful FK■ordered deletes).

`backend/enterprise_inventory_routes.py` — advanced stores logistics

- **Plain English:** The "warehouse pro" features: leftover■material tracking, material issue slips, goods■receipt inspection, stock■audit counts, and spreadsheet import.
- **Technical:** `register(app)` adds endpoints for `remnants`, `material_issue_slips`, `goods_receipt_notes`/`grn_items`, `cycle_counts`/`cycle_count_items`, a variance report, and CSV import.

`backend/ai_copilot.py` — the optional AI assistant

- **Plain English:** A chatbot that answers factory questions and writes reports from your real data — switched on only when a client provides an AI key.
- **Technical:** `register(app)` adds `/ai/status`, `/ai/ask`, `/ai/report`. `_build_factory_context` compiles a token■efficient data snapshot; `_ask_claude` calls the Anthropic Messages REST API via stdlib `urllib` (no SDK). Gated on `ANTHROPIC_API_KEY`; model from `AI_MODEL` (default `claude-haiku-4-5`).

`backend/industrial_adapters.py` — the protocol bridge

- **Plain English:** The part designed to talk to real machine controllers over industrial languages; today it fakes them realistically.
- **Technical:** `PROTOCOLS` registry, `ProtocolAdapter` base + `SimulatorAdapter`, `get_adapter`, `seed_industrial` (one demo device per protocol), `tick_industrial` (polls one online device -> `IndustrialSignal` rows, bounded), and `GET /industrial/protocols`. Real drivers plug in on an edge agent by overriding `read()`.

4.4 Backend — simulators & utility scripts (run by hand, not part of the web app)

File	Plain English	Technical
`phase30_plc_simulator.py`	Pretends to be a real PLC, broadcasting live machine telemetry over MQTT.	Publishes JSON payloads to the MQTT broker so `mqtt_service.py` ingests them via the real pipeline.
`mqtt_machine_publisher.py`	An earlier/simpler MQTT machine■telemetry publisher for testing.	Standalone paho■mqtt publisher to `flowmes/machines`.
`mqtt_listener.py`	A standalone MQTT debug listener.	Subscribes and prints incoming messages.
`live_simulator.py`	Older standalone live■data simulator (predecessor of `factory_simulator.py`).	Loops and writes rows directly.
`reseed_inventory.py`	Resets/reloads inventory demo data.	Deletes and re■seeds inventory rows.
`reset_machines.py`	Resets machine rows to a clean state.	Utility DB script.
`generate_pdf.py` *(repo root)*	Builds the printable "FlowMES Complete Guide" PDF walkthrough.	Uses a PDF library to render `docs/FlowMES_Complete_Guide.pdf`.

Staging / not■yet■merged files (present but not wired into the running app)

These are development artifacts kept for reference — they are *not* imported by `main.py` and don't affect the running app: `main_predictive_endpoint_to_add.py`, `phase11_routes_to_merge.py`, `phase11_model_to_add.py`, `phase11_schema_to_add.py`. The `*_to_add`/`*_to_merge`/`phase11_*` naming marks code snippets staged during phased development. *(Note: `analytics_engine.py` and `predictive_engine.py` are active — imported by `main.py` and documented in 4.2 — despite the "engine" name.)*

4.5 Backend — configuration

File	Purpose
`requirements.txt`	The list of Python libraries the backend needs (FastAPI, SQLAlchemy, paho■mqtt, python■jose, bcrypt, psycopg2, websockets, sentry). Railway installs these.
`railway.toml`	Tells Railway how to build (NIXPACKS + `pip install`) and run (`uvicorn main:app`) the backend, and where to health■check (`/docs`).
`.env` *(not committed)*	Holds secrets locally: `DATABASE_URL`, `SECRET_KEY`, `MQTT_BROKER`, etc.

4.6 Frontend — pages (what the user navigates to)

4.7 Frontend — the `lib/` folder (shared helpers & types)

File	Purpose
`lib/api.ts`	The **API client** — one place that talks to the backend. `apiGet/apiPost/apiPatch/apiDelete` attach the JWT and handle errors; `getUserRole()` reads the role out of the token; `API_URL` comes from `NEXT_PUBLIC_API_URL`.
`lib/modules.ts`	The **module & permission map**. `NAV_ITEMS` (all ~29 menu items grouped into 5 packs), `PLAN_MODULES` (which packs each subscription plan unlocks), and `canRoleSeeView()` (which role sees which screen — Operator = shop■floor only, Supervisor = most, Admin = all, cross■tenant SaaS admin = founder■only).
`lib/live.ts`	The **WebSocket client** — connects to `/ws/live` and feeds live machine updates into the dashboard.
`lib/types.ts`	Core shared TypeScript types (e.g. the `Role` type, `User`).
`lib/utils.ts`	Small shared helper functions (formatting, etc.).
`lib/phase8-types.ts … phase30-types.ts`, `lib/mega-pack1■3-types.ts`	**Per■module type definitions.** Each file describes the exact shape of the data for one feature phase (e.g. `phase17-types` = customer orders, `phase18-types` = purchasing, `mega-pack3-types` = SaaS admin/costing/operator). They exist because the app was built in phases; think of them as the "labels" that keep frontend and backend data in sync.

4.8 Frontend — the `components/` folder (~50 module screens)

The dashboard is assembled from many self■contained **"Section" components**, one per module. Each fetches its slice of data and renders the tables, forms and charts for that module. You don't need to read them all — the name tells you the module:

Core MES & analytics: `MachineSection`, `MachineStateSummary`, `MachineTimeline`, `DowntimeSection`, `ShiftSection`, `ShiftKPIs`, `AnalyticsSection`, `OEETrendCharts`, `ExecutiveOeeSection`, `ManagementDashboard`, `KpiCard` (a reusable stat tile).

Operations: `WorkOrdersSection`, `BomViewer`, `ProductionSection`, `ProductionPlanSection`, `SchedulingSection`, `OperatorTerminalSection`, `OrdersDispatchSection`.

Factory operations: `InventorySection`, `EnterpriseInventory`, `GmatsInventory`, `PurchasingSection`, `QualitySection`, `MaintenanceSection`, `DocumentsSection`, `CostingSection`, `DigitalTwinSection`.

Intelligence: `PredictiveMaintenanceSection`, `AIInsightsSection`, `AICopilot`, `IoTCommandSection`, `IndustrialConnectivity`, `IndustrialGatewaySection`, `EscalationSection`, `AlertsSection`, `NotificationsSection`, `ReportsSection`.

Admin & platform: `SaaSAdminSection`, `UsersSection`, `EnterprisePolishSection`, `Header`, `Sidebar`, `LockedModuleView` (the "upgrade to unlock" placeholder for locked plans), `Phase28UiRefactorNote`.

- **Technical note:** these are React function components using the `lib/api.ts` client and `lib/*-types.ts` types; the parent `dashboard/page.tsx` passes each one its data plus role■gated action callbacks (e.g. `deleteX` is only

passed to Admins).

4.9 Documentation & other files

File	Purpose
`README.md`	Project overview, features, stack, phase roadmap, and local■run instructions.
`docs/GMATS-Pilot-Proposal.md`	The paid■pilot proposal for the GMATS client (Rs pricing placeholders).
`docs/Production-Setup.md`	The go■live checklist (DB backups, monitoring, custom domain, secret rotation).
`docs/FlowMES_Complete_Guide.pdf`	A printable end■to■end walkthrough guide.
`docs/FlowMES-Complete-Documentation.md`	**This document.**

Part 5 — How to run, deploy and extend it

5.1 Run it on your own computer

```
# 1) Backend (Python API)
cd backend
python -m venv venv && venv\Scripts\activate      # create + enter a virtual environment
pip install -r requirements.txt                   # install libraries
python -m uvicorn main:app --reload               # start the API at http://127.0.0.1:8000
#     ->  interactive API docs live at http://127.0.0.1:8000/docs

# 2) Frontend (the dashboard)
cd frontend
npm install
npm run dev                                        # start the UI at http://localhost:3000

# 3) (optional) Live machine simulation over MQTT — needs a broker on 127.0.0.1:1883
cd backend
python phase30_plc_simulator.py
```

On first start the backend auto■creates the tables and seeds a full demo factory, so the dashboard has data immediately.

5.2 Deploy it (already set up)

- Push to `master` -> the **backend redeploys on Railway** and the **frontend redeploys on Vercel**, automatically.
- Set secrets as **environment variables** on Railway (`DATABASE_URL`, `SECRET_KEY`, `ALLOWED_ORIGINS`, optional `ANTHROPIC_API_KEY`, `SENTRY_DSN`, `GMATS_ADMIN_PASSWORD`) — never in code.
- ! Do **not** add a `Dockerfile` in `backend/` — Railway would use it instead of NIXPACKS and deploys would silently fail.

5.3 Add a new module (the pattern)

1. **Backend:** add the table(s) to `models.py`, the request/response shapes to `schemas.py`, and either add endpoints in `main.py` or create a new `xyz_routes.py` with a `register(app)` function and call it once in `main.py`.

2. **Frontend:** add a `lib/xyz-types.ts`, a `components/XyzSection.tsx`, a `NAV_ITEMS` entry in `lib/modules.ts` (choosing its module pack + role visibility), and render it in `dashboard/page.tsx`.

5.4 Onboard a new client (make it multi■tenant)

1. Add the client to `CLIENT_TENANTS` in `main.py` (username -> tenant code).
2. Create their users with the right `tenant_code`.
3. Set their licence/branding in `TenantConfig` (via the platform■owner endpoints). They now log in, see only their data, their logo, and their unlocked modules.

Appendix A — Full API reference (every endpoint)

The backend exposes ~150 endpoints. Almost every module follows the same **CRUD + analytics** pattern:

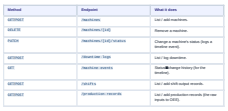
```
GET    /things             ->  list all
POST   /things             ->  create one
PATCH  /things/{id}        ->  update one
DELETE /things/{id}        ->  delete one
GET    /analytics/things   ->  summary/KPIs for that module
```

Conventions used below: unless noted, every endpoint needs a valid login token (`Authorization: Bearer <JWT>`). `Admin` / `Admin+Sup` marks endpoints restricted to those roles. `(public)` marks public (no login) endpoints. The always■current, interactive version of this reference is auto■generated by FastAPI at **`/docs`** (Swagger UI).

Authentication & users

Method	Endpoint	What it does
`GET`	`/` (public)	Liveness ping — returns "FlowMES Backend Running".
`GET`	`/me`	The current user, decoded from the token.
`POST`	`/register` (public)	Bootstrap the very first Admin (blocked once any user exists).
`POST`	`/login` (public)	Log in; returns the JWT, role and tenant.
`GET`	`/users` Admin	List employees in the caller's company.
`POST`	`/users` Admin	Add an employee (with role).
`PATCH`	`/users/{id}/role` Admin	Change a user's role.
`PATCH`	`/users/{id}/password` Admin	Reset a user's password.
`DELETE`	`/users/{id}` Admin	Remove a user.

Machines & shop floor



Analytics, OEE & alerts

Method	Endpoint	What it does
GET	/oee/summary	Factory■wide OEE snapshot.
GET	/analytics/summary	Overview■dashboard KPIs.
GET	/analytics/oee-trends	OEE trend series for the charts.
GET	/analytics/machine-timeline	Per■machine status timeline.
GET	/analytics/machine-state-summary	Time spent Running/Idle/Down per machine.
GET	/analytics/shift-kpis	Shift efficiency (actual vs target).
GET	/analytics/management	Executive management summary (top loss, worst machine, cost).
GET	/analytics/executive-oee	Deep executive OEE breakdown.
GET	/analytics/predictive-maintenance	Per■machine failure■risk scores.
GET	/analytics/system-health	System/data health metrics.
GET	/analytics/final-executive-summary	Single top■level executive summary.
GET	/alerts	Current dynamic alerts.
GET	/alerts/smart	Rule■based "smart" alerts.

Work orders, plans, schedules, operator & customer orders

Method	Endpoint	What it does
GET/POST	/work-orders	List / create work orders.
PATCH	/work-orders/{id}	Update a WO — marking **Completed** auto■moves BOM inventory.
DELETE	/work-orders/{id} Admin	Delete a WO.
GET	/bom	The Bill■of■Materials recipe book.
GET	/analytics/work-orders	Work■order analytics.
GET/POST/PATCH/DELETE	/production-plans[/{id}]	Production plan CRUD.
GET	/analytics/production-plans	Plan analytics.
GET/POST/PATCH/DELETE	/production-schedules[/{id}]	Schedule CRUD.
GET	/analytics/production-schedules	Schedule analytics.
GET/POST/PATCH/DELETE	/operator/executions[/{id}]	Operator■job CRUD (the operator terminal).
GET	/analytics/operator-terminal	Operator analytics.
GET/POST/PATCH/DELETE	/customer-orders[/{id}]	Customer■order CRUD.
GET	/analytics/customer-orders	Orders & dispatch analytics.
POST	/customer-orders/generate-late-order-escalations	Auto■raise late■order escalations.

Inventory, suppliers, purchasing

Method	Endpoint	What it does
GET/POST/PATCH/DELETE	/inventory/items[/{id}]	Inventory■item CRUD.
GET/POST	/inventory/transactions	Record/list stock movements.
GET	/analytics/inventory	Inventory analytics.
POST	/inventory/generate-low-stock-escalations	Auto low■stock escalations.
GET/POST/PATCH/DELETE	/suppliers[/{id}]	Supplier CRUD.
GET/POST/PATCH/DELETE	/purchase-orders[/{id}]	Purchase■order CRUD.
GET	/analytics/purchasing	Purchasing analytics.
POST	/purchase-orders/generate-overdue-escalations	Auto overdue■PO escalations.

Quality, maintenance, documents

Method	Endpoint	What it does
GET/POST/PATCH/DELETE	/quality/inspections[/{id}]	Quality■inspection CRUD.
GET	/analytics/quality	Quality analytics.
POST	/quality/generate-defect-escalations	Auto defect escalations.
GET/POST/PATCH/DELETE	/maintenance/tasks[/{id}]	Maintenance■task (CMMS) CRUD.
GET	/analytics/maintenance	Maintenance analytics.
POST	/maintenance/generate-overdue-escalations	Auto overdue■maintenance escalations.
GET/POST/PATCH/DELETE	/documents[/{id}]	Compliance■document CRUD.
GET	/analytics/documents	Document analytics.
POST	/documents/generate-review-escalations	Auto document■review escalations.

Escalations, notifications, reports, audit

Method	Endpoint	What it does
GET/POST/PATCH/DELETE	/escalations[/{id}]	Escalation CRUD.
POST	/escalations/from-smart-alerts	Convert smart alerts into escalations.
GET	/analytics/escalations	Escalation analytics.
GET/POST/PATCH	/notifications[/{id}]	List / create / mark■read notifications.
POST	/notifications/generate-system-notifications	Auto-system notifications.
GET/POST	/reports	Report■request log.
GET	/reports/downtime.csv	Downtime CSV export.
GET	/reports/shifts.csv	Shifts CSV export.
GET	/reports/oee.csv	OEE CSV export.
GET	/reports/daily-summary.txt	Daily intelligence report (plain text).
GET	/reports/intelligence-summary.txt	Intelligence summary (plain text).
GET/POST	/audit-logs	The audit trail of who did what.

Factory layout / digital twin

Method	Endpoint	What it does
`GET`/`POST`/`PATCH`/`DELETE`	`/factory-layout/nodes[/{id}]`	Layout■node CRUD (the visual factory map).
`POST`	`/factory-layout/auto-generate`	Auto■arrange the layout from machines.
`GET`	`/analytics/factory-command-center`	Command■center analytics.

AI copilot & recommendations

Method	Endpoint	What it does
`GET`/`PATCH`	`/ai/recommendations[/{id}]`	List / update AI recommendations.
`POST`	`/ai/generate-recommendations`	Generate rule■based recommendations.
`GET`	`/analytics/ai-insights`	AI■insights analytics.
`GET`	`/ai/status`	Is the Claude copilot connected?
`POST`	`/ai/ask`	Ask the copilot a question *(needs `ANTHROPIC_API_KEY`)*.
`POST`	`/ai/report`	Generate a daily AI management report *(needs key)*.

Industrial IoT & connectivity

Method	Endpoint	What it does
`GET`/`POST`	`/iot/telemetry`	IoT telemetry read/write.
`GET`	`/analytics/iot-command`	IoT command■center analytics.
`GET`/`POST`/`PATCH`	`/industrial/devices[/{id}]`	Industrial■device CRUD.
`GET`/`POST`	`/industrial/signals`	Live industrial signal read/write.
`GET`/`POST`	`/industrial/mappings`	PLC■signal -> MES■field mappings.
`GET`	`/analytics/industrial-gateway`	Gateway analytics.
`GET`	`/industrial/protocols`	The supported■protocol registry.
`WS`	`/ws/live`	**WebSocket** stream of live factory updates.

Platform / SaaS / multi■tenancy

Method	Endpoint	What it does
`GET`	`/health` (public)	Public health check (for uptime monitors).
`GET`	`/tenant-config`	The caller's own licence + branding.
`PATCH`	`/tenant-config` Admin	Re■brand your own workspace.
`GET`	`/tenant-configs` Founder	Every tenant's config (platform owner only).
`PATCH`	`/tenant-configs/{code}` Founder	Licence/brand any client.
`GET/POST/PATCH/DELETE`	`/saas/tenant[/{id}]`	Company■tenant (billing) CRUD.
`GET`	`/analytics/saas`	SaaS analytics.
`GET/POST/PATCH/DELETE`	`/cost-records[/{id}]`	Cost■record CRUD.
`GET`	`/analytics/costing`	Costing analytics.

GMATS client inventory (tenant■scoped, `/gmats/*`)

The real client workflow from Part 3.9. Representative endpoints (all GMATS■tenant■locked; exact set lives in `gmats_inventory_routes.py`): | Method | Endpoint | What it does | |---|---|---| | `GET`/`POST` | `/gmats/items` | List / add 4■bucket items. | | `POST` | `/gmats/stock-in` | Add physical stock. | | `POST` | `/gmats/items/{id}/correct` Admin | Directly correct a count (undo an operator mistake). | | `GET`/`POST` | `/gmats/proformas` | Quotations that **reserve** stock. | | `POST` | `/gmats/invoices` | Tax invoice — deducts physical, clears the reserve (a sale). | | `DELETE` | `/gmats/invoices/{id}` Admin | Void an invoice -> restore stock. | | `POST` | `/gmats/min` | Material Issue Note — free spares (deduct, not billed). | | `DELETE` | `/gmats/min/{id}` Admin | Void a MIN -> restore the issued spares. | | `POST` | `/gmats/import-csv` Admin | Import inventory from a Tally/Excel CSV. |

Enterprise stores logistics (`enterprise_inventory_routes.py`)

Advanced warehouse endpoints for **remnants** (leftover material), **material issue slips**, **goods■receipt notes** (GRN + inspection), **cycle counts** (stock audits) and a **variance report**, plus CSV import.

Appendix B — Frontend component reference (every screen)

The dashboard (`app/dashboard/page.tsx`) is one big menu that swaps in a different **Section** component depending on the selected module. Here is what each component is and renders.

Core MES & analytics

Component	What it renders
`KpiCard`	A reusable stat tile (a big number + label) used across dashboards.
`MachineSection`	The machine list with live status, utilization and controls to add/remove machines.
`MachineStateSummary`	A breakdown of how long each machine spent Running/Idle/Down.
`MachineTimeline`	A time■line strip of each machine's status changes through the day.
`DowntimeSection`	The downtime log with reasons and a form to record new downtime.
`ShiftSection`	Shift output records (target vs actual) and entry form.
`ShiftKPIs`	Shift efficiency KPI cards.
`AnalyticsSection`	The general analytics overview (KPIs + charts).
`OEETrendCharts`	Line/bar charts of OEE, Availability, Performance, Quality over time.
`ExecutiveOeeSection`	The executive OEE view for management.
`ManagementDashboard`	The high■level management summary (top loss reason, worst machine, cost of downtime).

Operations

Component	What it renders
`WorkOrdersSection`	The work■order list, create form, and status controls (drives BOM movement on completion).
`BomViewer`	The Bill■of■Materials recipe book (Admin■only).
`ProductionSection`	Production records / output view.
`ProductionPlanSection`	Production planning (what to make, when, on which machine).
`SchedulingSection`	The shift/machine schedule board.
`OperatorTerminalSection`	The operator's own screen to start jobs and log good/reject counts.
`OrdersDispatchSection`	Customer orders and dispatch progress.

Factory operations

Component	What it renders
`InventorySection`	The standard inventory module (items, stock, transactions, low■stock alerts).
`EnterpriseInventory`	Advanced stores logistics (remnants, issue slips, GRN, cycle counts).
`GmatsInventory`	The GMATS client's 4■bucket inventory + proforma/invoice/MIN workflow.
`PurchasingSection`	Suppliers and purchase orders.
`QualitySection`	Quality inspections (pass/fail, defects, rework, scrap).
`MaintenanceSection`	The CMMS — preventive/corrective maintenance tasks.
`DocumentsSection`	Compliance documents (SOPs, quality plans, records).
`CostingSection`	Cost records and costing analytics.
`DigitalTwinSection`	A visual factory■layout map (the "digital twin").

Intelligence

Component	What it renders
`PredictiveMaintenanceSection`	Per■machine failure■risk scores and recommendations.
`AIInsightsSection`	Rule■based AI recommendations.
`AICopilot`	The chat panel for the optional Claude copilot (ask questions, get reports).
`IoTCommandSection`	The IoT telemetry command center.
`IndustrialConnectivity`	The industrial■protocol connectivity screen (devices, protocols).
`IndustrialGatewaySection`	The industrial gateway (device/signal mappings).
`EscalationSection`	The escalation tracker (assign, resolve).
`AlertsSection`	The live alerts feed.
`NotificationsSection`	The notifications inbox.
`ReportsSection`	Report generation and CSV/text exports.

Admin & platform

Component	What it renders
`SaaSAdminSection`	The platform■owner view — manage client companies, plans and branding.
`UsersSection`	Employee management (add users, change roles, reset passwords).
`EnterprisePolishSection`	The "enterprise" showcase panel (audit log, system health, white■label).
`Header`	The top bar (logged■in user, company switcher).
`Sidebar`	An alternative sidebar navigation component.
`LockedModuleView`	The " upgrade to unlock" placeholder shown for modules the plan doesn't include.
`Phase28UiRefactorNote`	A small developer note component from the phase■28 UI refactor.

Glossary — every technical word, in plain English

Term	Plain■English meaning
MES	Manufacturing Execution System — software that runs and monitors a factory floor.
OEE	Overall Equipment Effectiveness — the master efficiency score (Availability × Performance × Quality).
SME	Small/Medium Enterprise — a smaller company (FlowMES's target customer).
SaaS	Software■as■a■Service — software you rent and use through a browser, not install.
Frontend / Backend	The part you see (browser) vs. the brain on the server.
API	Application Programming Interface — the set of web addresses the frontend calls to get/change data.
Endpoint	One specific API address, e.g. `GET /machines`.
Database / table / row	The permanent memory; a table is like a spreadsheet tab; a row is one record.
ORM (SQLAlchemy)	A translator that lets code use the database with objects instead of raw SQL.
PLC	Programmable Logic Controller — the small computer that controls a physical machine.
MQTT	A lightweight "post office" protocol machines use to broadcast their status.
WebSocket	A permanent open connection so the server can push live updates to the browser.
JWT	JSON Web Token — a tamper■proof digital ID badge issued at login.
bcrypt	A safe, deliberately slow way to scramble (hash) passwords.
RBAC	Role■Based Access Control — permissions based on your role (Admin/Supervisor/Operator).
Multi■tenancy	One app serving many companies, each isolated (identified by `tenant_code`).
White■label	Showing each company its own brand name/logo/colours.
BOM	Bill of Materials — the recipe of materials a product needs.
Proforma / Tax Invoice / MIN	A price quote / a real billed sale / a free■spares delivery note (GMATS workflow).
CORS	Browser security rule controlling which websites may call the API.
Vercel / Railway	The cloud hosts for the frontend / backend.
NIXPACKS	Railway's automatic app■builder.